

OpenAI's Guide to Function Calling in o3/o4-mini



Read more



Introduction

The o3/o4-mini models are the latest in our o-series of models trained to think for longer before responding. They are the smartest models we've released to date and represent a significant step forward from o1/o3-mini in tool calling capabilities. These models are trained to use tools natively within their chain of thought (CoT) which unlocks improved reasoning capabilities around when and how to use tools. We've released a guide on how to call functions with these models via the responses API, this guide builds on top of that and tells you how you can get the best function calling performance with these models.



Prompt guidance for better function calling performance

To fully utilize function calling intelligence behind o3/o4-mini models, we recommend a few best practices in both developer prompts and function descriptions.

A quick note on developer prompt, system prompt, and function descriptions for reasoning models

We introduced developer messages to make it explicit to reasoning models that an instruction is coming from the developer. In o-series models, any system message provided by the developer is automatically converted to a developer message internally. For practical purposes, you can treat the developer prompt as analogous to the traditional system prompt—but for clarity and correctness, this guide refers to all such instructions as developer prompts/messages.

When we refer to a function description in this document, we mean the explanatory text in the description field of each function object inside the tool parameter of an API request. This description tells the model when and how to use the function. Here's an example from our function calling documentation:

```
tools = [{  
    "type": "function",  
    "name": "get_weather",  
    "description": "Get current temperature for provided  
coordinates in celsius.",  
    "parameters": {  
        "type": "object",  
        "properties": {  
            "latitude": {"type": "number"},  
            "longitude": {"type": "number"}  
        },  
        "required": ["latitude", "longitude"],  
        "additionalProperties": False  
    },  
    "strict": True  
}]
```

Here, "Get current temperature for provided coordinates in celsius." serves as the function description.

Now that we got definitions out of the way, we can start getting into best practices.

Context setting via developer message

1. General context: In line with general prompt engineering best practices, role prompting is helpful in setting the base behavior, tone and outlining the set of actions that are possible. For example:

You are an AI retail agent.

As a retail agent, you can help users cancel or modify pending orders, return or exchange delivered orders, modify their default user address, or provide information about their own profile, orders, and related products.

2. Function Call ordering: o3/o4-mini are trained to accomplish goals with tools. However, it can make mistakes in the order of the tool calls. To guard against these cases, it is recommended to explicitly outline the orders to accomplish certain tasks. For example, to guard against the failure case that a coding agent possibly making a file in a directory that does not yet exist, adding the following will usually suffice:

check to see if directories exist before making files

For high volume and well defined tasks, we can make it even more robust by outlining the sequence of function to call explicitly, for example:

To Process a refund for a delivered order, follow the following steps:

- 1. Confirm the order was delivered. Use: `order\_status\_check`**
- 2. Check the refund eligibility policy. Use: `refund\_policy\_check`**
- 3. Create the refund request. Use: `refund\_create`**
- 4. Notify the user of refund status. Use: `user\_notify`**

3. Defining boundaries on when to use tools: It is helpful to clarify the model boundaries on when and when not to invoke certain tools. This can be done both at the developer prompt level and at the tool description level. Here is an example developer prompt:

Be proactive in using tools to accomplish the user's goal. If a task cannot be completed with a single step, keep going and use multiple tools as needed until the task is completed. Do not stop at the first failure. Try alternative steps or tool combinations until you succeed.

- Use tools when:

- The user wants to cancel or modify an order.**
- The user wants to return or exchange a delivered product.**
- The user wants to update their address or contact details.**
- The user asks for current or personalized order or profile info.**

- Do not use tools when:

- The user asks a general question like “What’s your return policy?”**
- The user asks something outside your retail role (e.g., “Write a poem”).**

If a task is not possible due to real constraints (For example, trying to cancel an already delivered order), explain why clearly and do not call tools blindly.

Function Description

A function’s description should clarify when it should be invoked and how its arguments should be constructed.

A function’s description is the ideal place to clarify both when the function should be invoked and how its arguments should be constructed. This serves as a durable interface contract between reasoning models and tool APIs.

In general, the function description defines what it does, how to invoke it. Developer instructions provide guidance to the agent using the tools. So if there are multiple tools that could be used for a similar purpose, the developer can disambiguate between them in the instructions. If the agentic workflow requirements have a preference for using tools in a specific order, or use certain tools frequently vs sparingly these would also go into the developer instructions.

A well-structured description can improve accuracy and reduce misfires by anchoring key criteria and argument requirements early. It also allows developers to encode “proactiveness” control heuristics outside the developer prompt, closer to the tool definition itself.

1. Usage Criteria: Similar to how you can refine function calling proactiveness through the developer prompt, you can further refine how a function gets called at the function description level. Here is an example for a `file_create` function:

Creates a new file with the specified name and contents in a target directory. This function should be used when persistent storage is needed and the file does not already exist.

- Only call this function if the target directory exists. Check first using the ``directory_check`` tool.

- Do not use for temporary or one-off content—prefer direct responses for those cases.

- Do not overwrite existing files. Always ensure the file name is unique.

- Do not overwrite existing files.

If replacement is intended and confirmed, use ``file_delete`` followed by ``file_create``, or use ``file_update`` instead.

2. Few shot prompting: While reasoning models do not benefit from few-shot prompting as much as non-reasoning models, we found that few shot prompting can improve tool calling performance, especially when the model struggles to accurately construct function arguments. For example, here is an example tool description for a `grep` tool passed in as tool description:

Use this tool to run fast, exact regex searches over text files using the `ripgrep` engine.

- Always escape special regex characters: () [] { } + * ? ^ \$ | . **
- Use `\\` to escape any of these characters when they appear in your search string.**
- Do NOT perform fuzzy or semantic matches.**
- Return only a valid regex pattern string.**

Examples:

Literal	→	Regex Pattern
function(	→	function\\(
value[index]	→	value\\[index\\]
file.txt	→	file\\.txt
user admin	→	user\\ admin
path\\to\\file	→	path\\\\to\\\\file

3. Key rules up front and minimize distractions: Note in the above example, the instruction to escape a special character is relatively the first thing the model reads.
A worse alternative would be:

Performs a fast regex-based text search that looks for exact pattern matches within files or entire directories, leveraging the ripgrep tool for high-speed scanning.

Output follows ripgrep formatting and can optionally display line numbers and matched lines.

To manage verbosity, results are limited to a maximum of 50 hits.

You can fine-tune the search by specifying inclusion or exclusion rules based on file types or path patterns.

This method is ideal when searching for literal text snippets or specific regular expressions.

It offers more accuracy than semantic methods when the goal is to locate a known string or structure.

It's generally recommended over semantic search when you're looking for a specific identifier—such as a function name, variable, or keyword—within a defined set of directories or file types.

This performs poorly because much of the prompt is not prescriptive and the most important rules for how to construct the argument are not front and center. The previous prompt scored 6% higher on a tool calling accuracy eval for using this ripgrep tool compared to the one above.

Guarding Against Function Calling Hallucinations

We are aware that the o3 model may be more prone to hallucinations than other models. These hallucinations may appear as the model promising to call tools in the background without actually doing so, or promising to call a tool in future turns, etc. In instances like these, it is helpful to be explicit in a few areas to minimize these types of hallucinations:

Explicit instructions: explicitly instruct the model to avoid common hallucinations like promising future function calls when it is not possible.

Do NOT promise to call a function later. If a function call is required, emit it now; otherwise respond normally.

2. Catch bad arguments early: Setting strict to true will ensure function calls reliably adhere to the function schema. We recommend turning it on whenever possible.

If your arguments have additional complex format requirements (e.g valid python code etc), adding the following instruction can remind the model of the expected format.

Validate arguments against the format before sending the call; if you are unsure, ask for clarification instead of guessing.

Avoid Chain of Thought Prompting

Since these models are reasoning models and produce an internal chain of thought, they do not have to be explicitly prompted to plan and reason between toolcalls. Therefore, a developer should not try to induce additional reasoning before each function call by asking the model to plan more extensively. Asking a reasoning model to reason more may actually hurt the performance.

A quick side note on reasoning summaries: the models will output reasoning tokens before calling tools. However, these will not always be accompanied by a summary, since our reasoning summaries require a minimum number of material reasoning tokens to produce a summary.

Responses API

Reasoning Items for Better Performance

We've released a cookbook detailing the benefits of using the responses API. It is worth restating a few of the main points in this guide as well. o3/o4-mini are both trained with its internal reasoning persisted between toolcalls within a single turn.

Persisting these reasoning items between toolcalls during inference will therefore lead to higher intelligence and performance in the form of better decision in when and how a tool gets called. Responses allow you to persist these reasoning items (maintained either by us or yourself through encrypted content if you do not want us to handle state-management) while Chat Completion doesn't.

Switching to the responses API and allowing the model access to reasoning items between function calls is the easiest way to squeeze out as much performance as possible for function calls.

Here is an the example in the cookbook, reproduced for convenience, showing how you can pass back the reasoning item using `encrypted_content` in a way which we do not retain any state on our end:


```
from openai import OpenAI
import requests
import json
client = OpenAI()
```

```
def get_weather(latitude, longitude):
    response = requests.get(f"https://api.open-meteo.com/v1/
forecast?latitude={latitude}&longitude={longitude}
&current=temperature_2m,wind_speed_10m&hourly=temperat
ure_2m,relative_humidity_2m,wind_speed_10m")
    data = response.json()
    return data['current']['temperature_2m']
```

```
tools = [{
    "type": "function",
    "name": "get_weather",
    "description": "Get current temperature for provided
coordinates in celsius.",
    "parameters": {
        "type": "object",
        "properties": {
            "latitude": {"type": "number"},
            "longitude": {"type": "number"}
        },
        "required": ["latitude", "longitude"],
        "additionalProperties": False
    },
    "strict": True
}]
```



```
context = [{"role": "user", "content": "What's the weather like in  
Paris today?"}]
```

```
response = client.responses.create(  
    model="o3",  
    input=context,  
    tools=tools,  
    store=False,  
    include=["reasoning.encrypted_content"] # Encrypted chain  
of thought is passed back in the response  
)
```

```
context += response.output # Add the response to the context  
(including the encrypted chain of thought)  
tool_call = response.output[1]  
args = json.loads(tool_call.arguments)
```

```
result = get_weather(args["latitude"], args["longitude"])
```

```
context.append({  
    "type": "function_call_output",  
    "call_id": tool_call.call_id,  
    "output": str(result)  
})
```

```
response_2 = client.responses.create(  
    model="o3",  
    input=context,  
    tools=tools,  
    store=False,  
    include=["reasoning.encrypted_content"]  
)
```

```
print(response_2.output_text)
```

The current temperature in Paris is about 18.8 °C.

Agentic Experience with Hosted tools.

Responses API supports a set of hosted/built-in tools. We recently also added new tools and features in the responses API which makes it easier to build agentic applications that connect to external services; With built-in tools in the Responses API, developers can create more capable agents with a single API call. You can mix and match hosted tools and custom tools in the same session. This unlocks powerful composition patterns, but it also makes tool routing clarity critical. Here are a couple of concrete recommendations:

1. Explicitly define tool usage boundaries in the developer prompt: If multiple tools can fulfill similar roles (e.g. both the python tool and a custom calculator), instruct the model which tool is preferred and when. This reduces ambiguity, improves accuracy, and avoids tool overuse or underuse. :

You are a helpful research assistant with access to the following tools:

- python tool: for any computation involving math, statistics, or code execution***
- calculator: for basic arithmetic or unit conversions when speed is preferred***

Always use the python tool for anything involving logic, scripts, or multistep math. Use the calculator tool only for simple 1-step math problems.

2. Clarify when internal knowledge is not sufficient: Even though o3/o4-mini models can often solve tasks on their own, tools may provide more reliable answers. Use the system prompt to steer the model away from “trying to solve it itself” when a tool is more appropriate.

You have access to a `code\_interpreter`. Always prefer using `code\_interpreter` when a user asks a question involving:

- math problems***
- data analysis***
- generating or executing code***
- formatting or transforming structured text***

Avoid doing these directly in your own response. Always use the tool instead.

3. Since the developer prompt acts as a centralized, durable contract, spell out decision boundaries for tools here when we want to mix and match hosted tools with your custom functions, including coverage overlap, confidence expectations, or fallback behavior:

Use `python` for general math, data parsing, unit conversion, or logic tasks that can be solved without external lookup—for example, computing the total cost from a list of prices.

Use `calculate\_shipping\_cost` when the user asks for shipping estimates, as it applies business-specific logic and access to live rate tables. Do not attempt to estimate these using the `python` tool.

When both could be used (e.g., calculating a delivery fee), prefer `calculate\_shipping\_cost` for accuracy and policy compliance. Fall back to `python` only if the custom tool is unavailable or fails.

4. More on MCP: We have a more detailed guide on best practices for using MCP tools, but for completeness, we will reiterate a few high-level guidelines here (these are not specific to o3/o4-mini, but are still relevant).

Filter tools to avoid ballooning payloads: take advantage of the `allowed_tools` parameter to use only the tools that are necessary and save on unnecessary context: Since you do not always need all of the tools returned by the MCP server, you can filter to only the necessary tools via the `allowed_tools` field.


```
"tools": [  
  {  
    "type": "mcp",  
    "server_label": "gitmcp",  
    "server_url": "https://gitmcp.io/openai/tiktoken",  
    "allowed_tools": ["search_tiktoken_documentation",  
"fetch_tiktoken_documentation"],  
    "require_approval": "never"  
  }  
]
```

Reduce latency via caching and reserve reasoning models for high complexity tasks: make sure you are either passing back `mcp_list_tools` or include `previous_response_id` to make sure the API does not need to reimport the list of tools again and again unnecessarily.

Use MCP with other tools: You can mix and match MCP with other hosted tools and your custom defined functions. If you are mixing the tools, it is helpful to define the decision boundaries and be explicit about when to use a tool over another using the overall developer prompt. Here is a great example from the MCP tool guide.